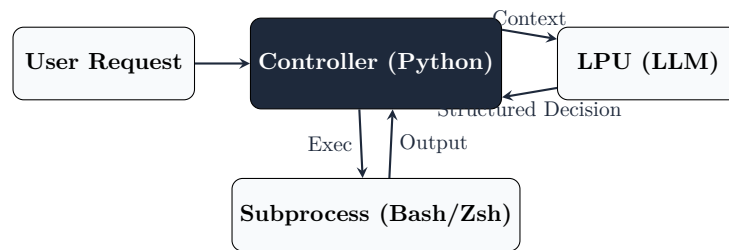


Assignment 3: Agentic Shell (doit)

LLM-Class 2025-2026



Authors:

Yuval Reuveni & Arbel Tepper

July 2026

1 Single command at a time.

The core requirement of this section is translating a single natural language request into a shell command, printing it, executing it in the user's active shell, and displaying the output.

1.1 Implementation Details

We developed the entry-point executable `doit` in Python (placed on the user's PATH via `~/.local/bin`). The system operates by assembling a system prompt, appending an environment block detailing the user's shell parameters, and calling the LPU with a command-step cap of `max_steps=1`.

If the user's prompt represents a command, the LPU calls the `run_command` tool, which is intercepted and executed via Python's `subprocess.run`. If the prompt represents a general inquiry, the LPU responds with the `answer` tool.

1.2 Design Decisions

Design Decision: Shell-Function Wrapper for Directory Navigation (`cd`)

Problem: When a user runs a Python script, it runs in a child subprocess. Any directory change (`cd`) executed by the script is lost when the script exits, returning the user to the original terminal directory.

Solution: We resolved this by writing a dual-shell wrapper. The `change_dir` tool verifies that the path exists and writes it to a temporary file: `~/.doit/cd_target_${DOIT_SESSION}`. Upon `doit`'s termination, a shell function wrapper intercepts the exit, checks for the presence of this file, runs the native `cd` command inside the parent shell, and deletes the file.

Design Decision: Dual-Shell Variable Normalization

Context: Since different team members use different operating systems and shells (e.g., macOS/Zsh vs. Linux/Bash), command arguments and behavior can diverge.

Solution: The Controller auto-detects the host shell via `$SHELL` and includes the shell type and operating system in the environment prompt block. The system prompt instructs the LPU to emit syntax tailored to the host environment.

Design Decision: Primary API Model Selection

Choice: We selected `openai/gpt-4o-mini` as our primary API model. It is inexpensive, runs with low latency, and features robust native tool-calling capabilities that simplify core development.

1.3 Example Interactions

Below are two successful single-command interactions run using our primary model:

```

1 $ doit "show me all files here, including hidden ones"
2 $ ls -la
3 total 232
4 drwxr-xr-x@ 6 yuval  staff    192 Jul  7 18:23 acdl
5 -rw-r--r--@ 1 yuval  staff  12405 Jul  7 18:24 CLAUDE.md
6 -rw-r--r--@ 1 yuval  staff  30675 Jul  7 18:57 DECISIONS.md
7 -rwxr-xr-x@ 1 yuval  staff   1319 Jul  7 15:00 doit

```

Listing 1: Listing files including hidden ones.

```

1 $ doit "how much disk space is left?"
2 $ df -h
3 Filesystem      Size   Used  Avail Capacity  iused ifree %iused  Mounted on
4 /dev/disk3s1    926Gi   17Gi   702Gi     3%    459k  4.3G    0%      /

```

Listing 2: Disk space command execution.

1.4 ACDL Context Specification: DoitAgentSingleCommand

The formal context structure representing the single command-at-a-time execution is detailed in the visual ACDL diagram below:

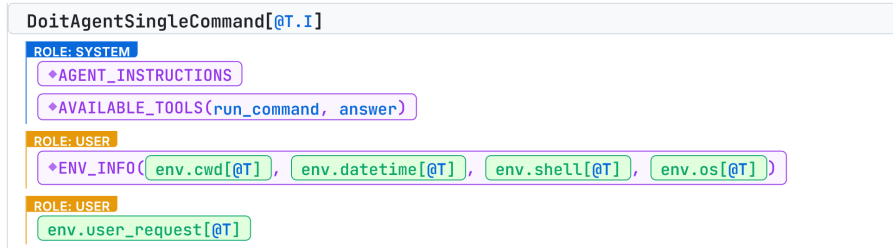


Figure 1: Visual representation of the ACDL context for DoitAgentSingleCommand.

2 Identifying dangerous commands

To prevent the agent from accidentally corrupting the host machine or executing destructive actions without oversight, we implement safety policies that intercept filesystem mutations.

2.1 Implementation Details

Safety is handled through a double-layer verification policy (defense in depth):

1. **LPU Self-Report (Layer 1):** The `run_command` tool schema requires the LPU to populate the `is_destructive` boolean flag and provide an **explanation** of what the command will do.
2. **Deterministic Guard (Layer 2):** The Controller intercepts the decision and passes the command through a pre-compiled regex guard in `safety.py`. If the regex matches mutating operations (e.g. `rm`, `mv`, `cp`, `mkdir`, redirects `>`, `>>`, `git reset`), it overrides a false “safe” report, forcing `is_destructive=True`.

2.2 Design Decisions

Design Decision: In-Band Self-Report vs. Separate LLM Judge Call

Alternative Rejected: We rejected using a separate LLM call to classify command destructiveness.

Rationale: A second LLM call doubles latency and API costs. Because the regex guard is required to handle safety deterministically anyway, we require the LPU to report destructiveness in the same JSON object as the command itself. The regex guard acts as the fallback mechanism. If the guard overrides the model's safety flag, the event is logged for analysis.

Safety Refusals vs. Confirmation Gates

Hard Refusals: Any command containing `sudo` or starting with an interactive full-screen program (e.g., `vim`, `nano`, `less`, `top`) is blocked immediately in the Controller. No confirmation is offered.

Confirmation Gates: When a command is flagged destructive (either by the model or the regex override), the Controller halts, displays the command and its explanation, and prompts the user with `Proceed? [y/N]`. Execution only proceeds if the user inputs `y` or `yes`.

2.3 Example Interactions

```

1 $ doit "delete junk.txt"
2 [WARNING] This command modifies the filesystem:
3     rm junk.txt
4     This command deletes the file junk.txt.
5 Proceed? [y/N] y
6 $ rm junk.txt

```

Listing 3: Handling a destructive command with user confirmation.

```

1 $ doit "open vim to write a story"
2 doit: 'vim' opens an interactive program doit can't wait for. Run it
   yourself:
3     vim

```

Listing 4: Interactive program refusal.

2.4 ACDL Context Specification: DoitAgentSafety

The formal context structure representing the safety gate integration is detailed in the visual ACDL diagram below:

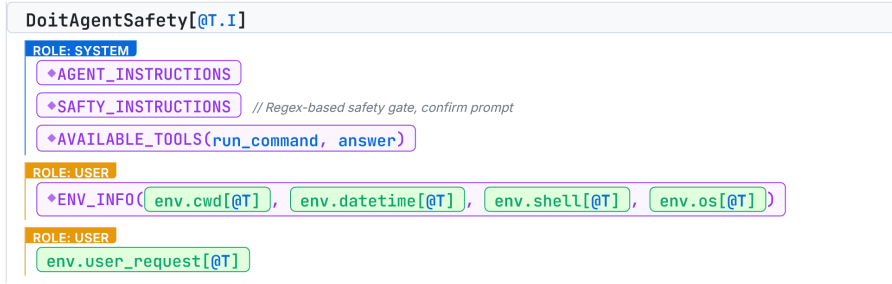


Figure 2: Visual representation of the ACDL context for DoitAgentSafety.

3 Model flexibility

To ensure the utility can run independently of commercial API keys, we extended the system to run local models locally.

3.1 Implementation Details

We introduced a configuration file `~/doit.cfg` to declare the active model and its adapter. We implement two code adapters behind a unified interface:

Design Decision: Native vs. Prompted Adapter Architectures

Native Adapter (`_call_native`): For models supporting LiteLLM’s JSON tool-calling protocol (e.g., `gpt-4o-mini`, `qwen3:4b-instruct`). LiteLLM passes tool schemas out-of-band and receives structured tool outputs.

Prompted Adapter (`_call_prompted`): For local models that do not natively support tool-calling (e.g., `gemma3:4b`). It renders tool schemas as text, appends them to the system prompt, and enforces a strict JSON wrapper. To handle parser failures, we implement a string-based balanced brace scanner to locate JSON blocks, and allow **one retry** by appending the syntax error back to the LPU.

3.2 Evaluation of Local Models

We evaluated three models using the test suite in `tests/cases.md`:

3.3 Analysis of Divergences and Failure Modes

3.3.1 Gemma-3 Impossible Request Failure

When asked to perform a physically impossible command like "make my laptop fly", the native models correctly responded with the conversational `answer` tool. However, the local `gemma3:4b` model (running under the prompted adapter) attempted to execute it via bash, choosing the `run_command` tool with:

Table 1: Model capabilities and adapters.

Model Name	Deployment	Adapter	Tool-Calling Capability
openai/gpt-4o-mini	API (Remote)	Native	Full structured tool support
ollama/qwen3:4b-instruct	Local (Ollama)	Native	Local tool-calling support
ollama/gemma3:4b	Local (Ollama)	Prompted	None; relies on text JSON prompts

```
1 echo "Sorry, I cannot make your laptop fly."
```

Because the model appended a trailing double quote, the command crashed with a shell parse error: `zsh:1: unmatched "`.

3.3.2 Prompted Adapter Recovery Logs

Under the prompted adapter, local models often return markdown fences and conversational preambles. Below is a log showing our parser successfully stripping the preamble and recovering the JSON payload:

```
1 Raw LPU Output:
2 Sure, here is the command to view disk space:
3 ```json
4 {
5   "tool": "run_command",
6   "args": {
7     "command": "df -h",
8     "is_destructive": false,
9     "explanation": "Check available disk space."
10  }
11 }
12 ```
13 Parser Action: Strip fences, run brace scan -> extracted {"tool": "
    run_command", ...}
```

Listing 5: JSON de-fencing and brace-scanning recovery.

3.4 ACDL Context Specification: DoitAgentPrompted

For models without native tool-calling, tool schemas move into the system prompt prose, and the raw text reply is parsed. The visual ACDL diagram below shows how we handle JSON validation and retries:

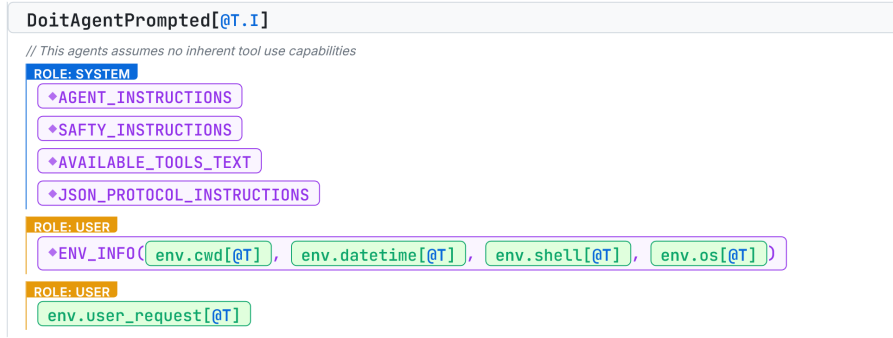


Figure 3: Visual representation of the ACDL context for DoitAgentPrompted.

4 Multi-turn

To move from isolated commands to conversational sequences, we carry state across invocations, allowing commands to refer to previous outputs.

4.1 Implementation Details

When `doit` runs, it loads the session ID from the environment variable `DOIT_SESSION` (which defaults to "default" if unset). The Controller reads and parses the JSONL session logs located at `~/.doit/sessions/$DOIT_SESSION.jsonl`, extracts the last $K = 10$ turns, and splices them back into the prompt context.

4.2 Design Decisions

Design Decision: JSONL Storage Format

Problem: Selecting an appropriate storage format for history.

Choice: We chose JSONL (JSON Lines) files over SQLite. JSONL is lightweight, human-readable, and allows $O(1)$ appends. It facilitates debugging and log evidence curation for the report.

Design Decision: Tiered Truncation Budgets (HOT vs. COLD)

Problem: Feeding complete command outputs (e.g. file listings or compiler logs) back into the prompt quickly overflows the context window.

Solution: We implement a tiered head/tail truncation strategy. The active turn's output is allocated a **Hot Budget** (up to 3,000 characters head, 1,000 characters tail) to support follow-up decisions. Older replayed history turns are truncated to a **Cold Budget** (1,000 characters head, 300 characters tail), which provides enough context to resolve follow-ups (like referring to listed files) while saving tokens.

4.3 Example Multi-Turn Interactions

Below is a live multi-turn session log using `gpt-4o-mini`, demonstrating how the model changes flags and behavior rather than parroting commands:

```

1 $ DOIT_SESSION=demo doit "list the files here"
2 $ ls -l
3 total 232
4 -rw-r--r--@  1 yuval  staff  12405 Jul  7 18:24 CLAUDE.md
5 -rw-r--r--@  1 yuval  staff  30675 Jul  7 18:57 DECISIONS.md
6
7 $ DOIT_SESSION=demo doit "now sort them by date"
8 $ ls -lt
9 total 232
10 -rw-r--r--@  1 yuval  staff  30675 Jul  7 18:57 DECISIONS.md
11 -rw-r--r--@  1 yuval  staff  12405 Jul  7 18:24 CLAUDE.md
12
13 $ DOIT_SESSION=demo doit "now in ascending order"
14 $ ls -ltr
15 total 232
16 -rw-r--r--@  1 yuval  staff  12405 Jul  7 18:24 CLAUDE.md
17 -rw-r--r--@  1 yuval  staff  30675 Jul  7 18:57 DECISIONS.md

```

Listing 6: Relational multi-turn listing and sorting.

4.4 ACDL Context Specification: DoitAgentHistory

The formal context structure representing the conversation history replay integration is detailed in the visual ACDL diagram below:



Figure 4: Visual representation of the ACDL context for DoitAgentHistory.

5 Clarifications

When a user request is ambiguous, the agent must ask clarifying questions rather than guessing or executing a potentially harmful action.

5.1 Implementation Details

We introduce the `ask_user` tool. When invoked by the LPU, the Controller prints the question alongside a numbered options menu, blocks on user input, and feeds the response back to the LPU. This process runs as a within-turn loop and does not count towards the command-step count.

5.2 Design Decisions

Design Decision: Unanswered Clarification Default Behavior

Problem: How to proceed if the user presses Enter (empty input) or interrupts the prompt.

Solution: Pressing Enter returns "no answer" to the LPU, instructing it to proceed with the default option it stated. If this default results in a destructive command, the Layer 2 safety confirmation gate (Proceed? [y/N]) still blocks execution. Pressing Ctrl-C raises a KeyboardInterrupt, aborts the turn immediately, and logs the abort status.

5.3 Non-Annoyance Cap

To prevent the agent from asking questions repeatedly, we implement two controls:

1. **System Prompt Policy:** The LPU is instructed to ask questions only when a wrong guess is destructive, or when interpretations lead to different results with no clear default. Otherwise, it must assume the most common interpretation and state it in the final output.
2. **Structural Cap:** We enforce a hard limit of `MAX_CLARIFICATIONS=2` in the Controller. If the LPU exceeds this, the Controller feeds a capped message warning the model to commit to its best guess.

5.4 Example Interaction

Below is a live interaction showing how the clarification loop and safety confirmation gate stack together cleanly:

```

1 $ doit "delete the logs"
2 Are you sure you want to delete the logs? This action cannot be undone.
3   1. Yes
4   2. No
5 Your answer (number or text): 1
6
7 [WARNING] This command modifies the filesystem:
8   rm *.log
9   This command deletes log files in the current directory.
10 Proceed? [y/N] y
11 $ rm *.log

```

Listing 7: Stacked safety: clarification leading to a destructive confirmation gate.

5.5 ACDL Context Specification: DoitAgentClarify

The final formal context structure for our agent, representing both replayed turns and the within-turn clarification loop, is detailed in the visual ACDL diagram below:

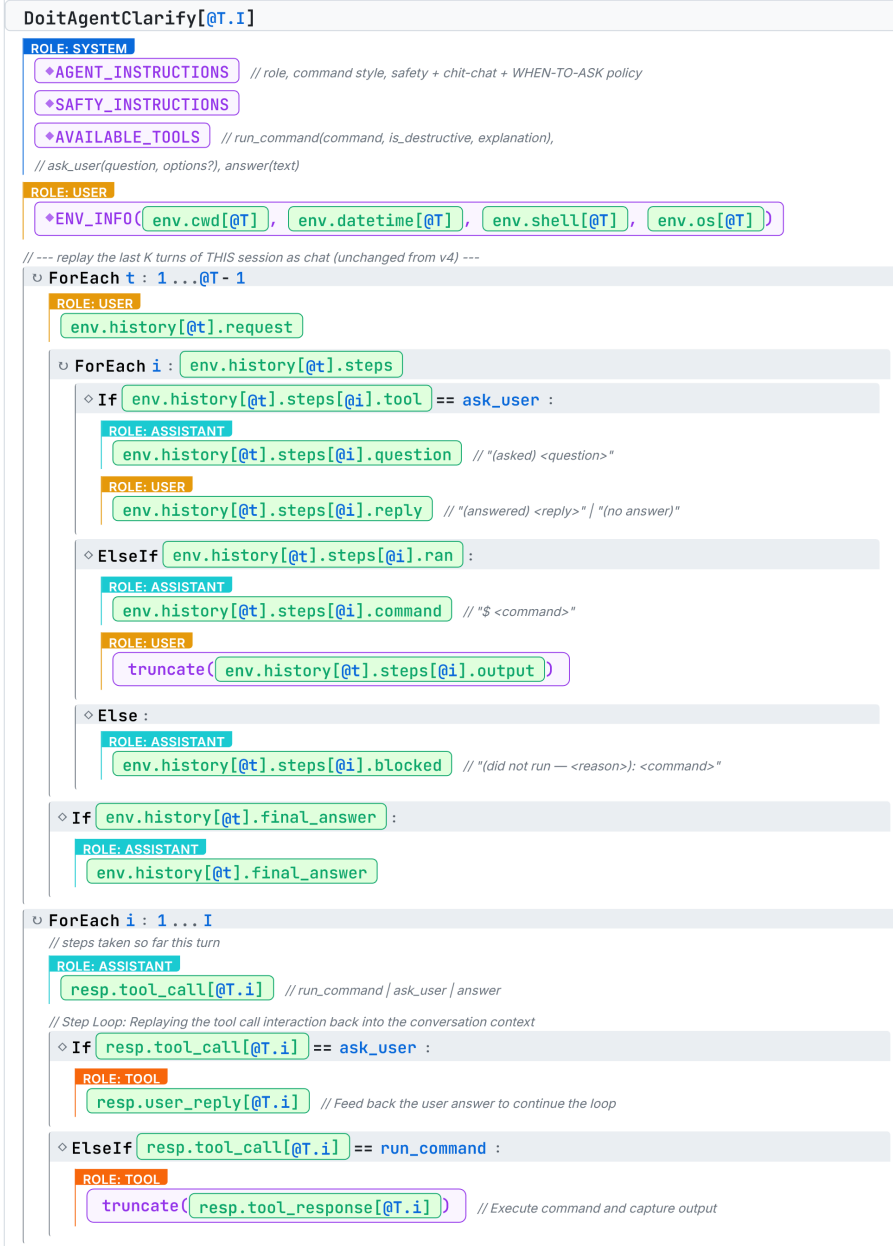


Figure 5: Visual representation of the ACDL context for DoitAgentClarify & Multi-turn).

6 Richer interactions

Richer interactions require the agent to respond to non-commands (e.g., “how do I do X?”) and handle follow-up refinement chains (e.g., “how do I list files?” → “modify it to show sizes” → “execute it”).

6.1 Implementation Details

We resolved this without introducing new tools or classification models.

- **Explaining vs. Running:** If a user asks a question, the LPU uses the `answer` tool. The Controller displays the answer and terminates the turn loop without executing commands.
- **Execution from History:** When the user subsequently requests “execute it”, the request is sent with the session’s turn history. The LPU reads the command structure it described in the previous `answer` and passes it to the `run_command` tool.

6.2 Example Interaction

Below is a three-turn interaction chain showing history extraction and execution:

```
1 $ doit "how do I find python files modified this week?"
2 You can use: find . -name "*.py" -mtime -7
3 $ doit "modify it to also show file sizes"
4 You can use: find . -name "*.py" -mtime -7 -ls
5 $ doit "execute it"
6 $ find . -name "*.py" -mtime -7 -ls
7 14283      8 -rw-r--r--    1 yuval  staff  1319 Jul  7 15:00 ./doit
```

Listing 8: How-do-I question, command modification, and execution.

6.3 Limitations

We observed that smaller local models (such as `gemma3:4b`) frequently fail on the third turn (“execute it”), responding with “Execute what?” or hallucinating unrelated commands. This indicates that tracking command references across multiple conversational turns is a capability limited to larger, instruction-tuned models.